

題目回饋

Requirement

1. List pharmacies, optionally filtered by specific time and/or day of the week.
 - 檢查放入奇怪參數是否會 4xx 以外的錯誤
 - 參數沒做任何檢查，造成 error
 - http code 都是 200
 - 是否檢查跨日
 - 沒做
 - 確認列出資料欄位的合理性
 - queryType 用 0 & 1 可讀性低
2. List all masks sold by a given pharmacy with an option to sort by name or price.
 - sql n+1 問題
 - 檢查放入奇怪參數是否會 4xx 以外的錯誤
 - 參數沒做任何檢查，造成 error
 - http code 都是 200
 - 會噴 500
3. List all pharmacies that offer a number of mask products within a given price range, where the count is above, below, or between given thresholds.
 - 檢查放入奇怪參數是否會 4xx 以外的錯誤
 - 參數沒做任何檢查，造成 error
 - 沒有檢查 $\text{priceMin} \leq \text{priceMax}$
 - 沒有檢查價格是否為負數
 - sql n + 1 問題
 - 效能議題 - 重複計算 Sum
 - 沒有分頁
4. Show the top N users who spent the most on masks during a specific date range.
 - sql n+1 問題
 - 無判斷開始時間 > 結束時間
5. Process a purchase where a user buys masks from multiple pharmacies at once.
 - 沒有使用 transaction
 - 沒有對客戶和藥局的餘額做動作
 - 效能議題，很多重複的 code
 - DateTime.Parse 無驗證
6. Update the stock quantity of an existing mask product by increasing or decreasing it.
 - http method 使用錯誤應該使用 patch
 - 沒有使用 transaction
 - 參數沒有驗證，資料庫可更改為負數
 - 相同的效能問題
7. (-3) Create or update multiple mask products for a pharmacy at once, including name, price, and stock quantity.

- 沒有使用 transaction
- n+1 問題
- 驗證邏輯效能問題 $O(n^2)$
- http method 使用錯誤應該使用 patch

8. Search for pharmacies or masks by name and rank the results by relevance to the search term.

- 分頁邏輯錯誤
 - Pharmacy/Mask case 做了兩次分頁，結果完全錯誤
 - totalCount 計算錯誤 → 永遠是 pageSize 或更少，分頁資訊錯誤

Design

Database

- db 關聯還行，但 system config 這張 table 感覺不必要
- open_time & close_time 使用 varchar 對 sql 不友善
- index 需要加強
- typo? `OPING_HOURS_ID`
- PHARMACY 中區分 `CASH_BALANCE` 和 `CURRENT_CASH_BALANCE` 有點奇怪也沒用到

API

- http method 錯誤
- API 設計不符合 RESTful 大多使用動詞 ex: `GetPharmaciesByMaskStock`
- 參數全部都沒有驗證，極其危險
- error 全部都是 200 不是 400 同時也沒使用 `ex.Message` 都是回覆“發生意外錯誤”
- request params 設置的合理性，很多 0 & 1 代替的值
- 會出現 500 error

Architecture

- 參數檢查的位置很怪，大多 controller 都沒處理
- Service 拆分的看不太懂 `HandleService` 和 `Service` 的職責混淆，`HandleService` 看起來應該是要處理一些特例的商業邏輯，但有些 `Service` 也包含業務邏輯 (ex: `PharmacyService.GetPharmacies`) `HandleService` 應該更專注於 Use Case
- 缺少 Repository 統一 response 規範

Implementation

code quality

readability

- 程式碼是否易讀
 - 程式碼判斷是太多，很多地方可以簡化與優化，太多簡寫 o & n etc...
- 太多的迴圈與重複的程式碼

performance

- 排序方法透過程式容易造成效能議題
- 很多的重複宣告與 sql method ex: `GetAll` & `ToListAsync` 使用狀況不佳的情況

maintainability

- 沒進行 db 版控

completion

- error handle 有待加強
- sql typo

大概整理：

PurchaseController.InsertPurchaseHistory - 交易完整性缺失

檔案: [T4U_Pharmacy_Service/HandleService/PurchaseHandleService.cs](#) (Line 45-110)

具體問題:

1. 沒有扣除客戶餘額

```
// 現況：只檢查餘額但不扣款
var customer = _customerService.Get(n => n.CustomerId == input.CustomerId);
if (customer.CurrentCashBalance < totalCash) {
    return "餘額不足";
}
// 沒有執行 customer.CurrentCashBalance -= totalCash;
```

2. 沒有增加藥局收入

```
// 沒有更新 pharmacy.CurrentCashBalance
```

3. 沒有 Transaction 控制

- 多個資料表操作 (PurchaseHistory, PharmacyMasksStockLog) 沒有包在交易中
- 若中途失敗會造成資料不一致

4. 沒有庫存扣減驗證

```
// 只記錄負數 log，但沒檢查庫存是否足夠
var log = new PharmacyMasksStockLog
{
    StockQuantity = -pharmacyOrder.Quantity // 可能超賣
};
```

SearchPharmacyMaskByKeyword - 分頁邏輯完全錯誤

檔案: [T4U_Pharmacy_Service/HandleService/PharmacyMaskHandleService.cs](#) (Line 98-176)

問題描述:

分頁功能完全無法正常運作，第 2 頁以後會回傳空陣列。

具體問題:

1. 重複分頁導致資料為空

```
// Pharmacy case (Line 133-151)

// 第一次分頁 (在資料庫)
var pharmaciesList = await _pharmacyService.GetAll()
    .Where(p => p.Name.Contains(keyword))
    .Skip((page - 1) * pageSize) // 例如 page=2, 跳過 5 筆
    .Take(pageSize)             // 取 5 筆 (第 6-10 筆)
    .ToListAsync();

results = pharmaciesList
    .Select(...)
    .OrderByDescending(...)
    .Skip((page - 1) * pageSize) // 又跳過 5 筆 !
    .Take(pageSize)             // 結果是空的
    .ToList();

// 實際情況：
// pharmaciesList 有 5 筆資料 (索引 0-4)
// Skip(5) 會跳過全部，結果回傳 0 筆
```

2. totalCount 計算錯誤

```
// Line 138
totalCount = pharmaciesList.Count; // 永遠是 pageSize 或更少
```

實際範例:

情境：資料庫有 100 筆藥局，搜尋 "藥" 符合 50 筆

Page 1 (pageSize=5):

- 資料庫取得第 1-5 筆 → pharmaciesList.Count = 5
- Skip(0).Take(5) → 回傳 5 筆 (正確)
- totalCount = 5 (應該是 50)

Page 2 (pageSize=5):

- 資料庫取得第 6-10 筆 → pharmaciesList.Count = 5
- 記憶體 Skip(5).Take(5) → 回傳 0 筆 (跳過全部)
- totalCount = 5

RESTful API 設計不符合規範

問題彙整:

Controller	現有路由	HTTP 方法	問題	建議路由
PharmacyController	/v1/Pharmacy/GetPharmacies	GET	動詞命名	/v1/pharmacies
PharmacyController	/v1/Pharmacy/GetPharmaciesByMaskStock	GET	動詞命名	/v1/pharmacies?maskId={id}&minStock={count}
MasksController	/v1/Masks/UpdateMaskQuantity	POST	應該用 PUT/PATCH	/v1/masks/{id} (PUT)
PharmacyMaskController	/v1/PharmacyMask/UpdateMaskData	POST	應該用 PUT	/v1/pharmacies/{id}/masks (PUT)
PurchaseController	/v1/Purchase/InsertPurchaseHistory	POST	動詞命名	/v1/purchases (POST)
CustomerController	/v1/Customer/GetTopCustomerPurchaseHistory	GET	動詞命名	/v1/customers/top-purchasers

具體問題:

1. 使用動詞而非名詞

```
/v1/Pharmacy/GetPharmacies    → /v1/pharmacies  
/v1/Masks/UpdateMaskQuantity → /v1/masks/{id}  
/v1/Purchase/InsertPurchaseHistory → /v1/purchases
```

2. HTTP 方法使用不當

- 更新操作使用 POST 而非 PUT/PATCH
- 沒有實作 DELETE 方法

3. URL 命名不一致

- 有的用複數 `Masks`
- 有的用單數 `PharmacyMask`
- 應該統一用複數

Controller 沒有檢查 Service 結果

問題:

```
// 所有 Controller 都直接回傳 200 OK  
[HttpGet]  
public async Task<ActionResult<PharmacyViewModel>> GetPharmacies(...)  
{  
    var result = await _pharmacyService.GetPharmacies(...);  
    return Ok(result); // 不管成功失敗都回傳 200  
}
```

重複的驗證邏輯 ($O(n^2)$)

檔案: `PharmacyMaskHandleService.cs` (Line 345-349)

問題:

```
// 複雜度  $O(n^2)$   
foreach(var updateMask in input.Masks)  
{  
    if(input.Masks.Where(n => n.MaskName == updateMask.MaskName).Count() > 1)  
    {  
        return string.Format("口罩不能重複，MaskName:{0}", updateMask.MaskName);  
    }  
}
```

建議:

```
// 使用 GroupBy - 複雜度  $O(n)$   
var duplicates = input.Masks  
    .GroupBy(m => m.MaskName)  
    .Where(g => g.Count() > 1)  
    .Select(g => g.Key)  
    .ToList();  
  
if (duplicates.Any())  
{
```

```
return $"口罩不能重複，MaskName:{string.Join(", ", duplicates)}";  
}
```

效能相關：

問題1: [T4U_Pharmacy_API/T4U_Pharmacy_Service/HandleService/PharmacyMaskHandleService.cs](#) 重複查詢，雖然最後用了 `ToListAsync()`，但前面的 `GetAll()` 已經觸發查詢

```
var pharmacies = _pharmacyMasksService.GetAll().Where(n => n.PharmacyId == pharmacyId);  
var list = await pharmacies.Select(n => new { n.Masks, n.Pharmacy, n.PharmacyMasksStockLogs, n.Price }).ToLis  
tAsync();
```

問題2:

[T4U_Pharmacy_API/T4U_Pharmacy_Service/HandleService/PharmacyMaskHandleService.cs](#)

`FirstOrDefault` 執行兩次。

應該可以：

```
var firstItem = list.FirstOrDefault();  
if (firstItem != null)  
{  
    obj.PharmacyId = firstItem.Pharmacy.PharmacyId;  
    obj.Name = firstItem.Pharmacy.Name;  
    obj.CashBalance = GetPharmacyCurrentCashBalance(pharmacyId);  
}
```

另外 `FirstOrDefault()` 可能回傳 `null` 直接壞掉

問題3: [T4U_Pharmacy_API/T4U_Pharmacy_Service/HandleService/PharmacyMaskHandleService.cs](#) 除了這個檔案外很多地方的排序都使用程式碼進行，當大量執行或者資料量較多時時記憶體會被塞滿

問題4:

[T4U_Pharmacy_API/T4U_Pharmacy_Service/Service/PurchaseHistoryService.cs](#)

```
foreach(var p in listPurchaseHistories)
```

每個客戶都要遍歷整個 `listDetail`，這樣時間複雜度是 $O(n^2)$

改成這樣會好一點

```
// 時間複雜度  $O(n)$   
var detailsByCustomer = listDetail  
    .GroupBy(d => d.CustomerId)  
    .ToDictionary(g => g.Key, g => g.ToList());  
  
foreach(var p in listPurchaseHistories)  
{  
    p.Details = detailsByCustomer.ContainsKey(p.CustomerId)  
        ? detailsByCustomer[p.CustomerId]  
        : new List<TransactionDetail>();  
}
```

問題： [T4U_Pharmacy_API/T4U_Pharmacy_Service/Service/PharmacyMasksService.cs](#)

效能議題 Between 情況下，每筆資料的 Sum 會計算 3 次（WHERE 2次 + SELECT 1次）

```
// Line 59-67
switch (quantityThresholdType)
{
    case QuantityThresholdType.Above:
        // 計算 Sum 一次
        pharmacies = pharmacies.Where(n => n.PharmacyMasksStockLogs.Sum(s => s.StockQuantity) >= quantityMin.Value);
        break;
    case QuantityThresholdType.Below:
        // 計算 Sum 一次
        pharmacies = pharmacies.Where(n => n.PharmacyMasksStockLogs.Sum(s => s.StockQuantity) <= quantityMax.Value);
        break;
    case QuantityThresholdType.Between:
        // 計算 Sum 兩次！
        pharmacies = pharmacies.Where(n =>
            n.PharmacyMasksStockLogs.Sum(s => s.StockQuantity) >= quantityMin.Value &&
            n.PharmacyMasksStockLogs.Sum(s => s.StockQuantity) <= quantityMax.Value);
        break;
}

// Line 82 - 又計算一次 Sum
StockQuantity = n.PharmacyMasksStockLogs.Sum(s => s.StockQuantity)
```

改成：

```
// 先計算 Sum，只算一次
var pharmacies = this.repository.GetAll()
    .Select(n => new
    {
        PharmacyMask = n,
        TotalStock = n.PharmacyMasksStockLogs.Sum(s => s.StockQuantity)
    });

// 使用已計算的值
switch (quantityThresholdType)
{
    case QuantityThresholdType.Between:
        pharmacies = pharmacies.Where(n =>
            n.TotalStock >= quantityMin.Value &&
            n.TotalStock <= quantityMax.Value);
        break;
}
```

問題： [T4U_Pharmacy_API/T4U_Pharmacy_Service/Service/PharmacyMasksService.cs](#)

一樣效能議題
Count() 會立即執行一次 SQL 查詢
ToListAsync() 又執行一次 SQL 查詢
總共執行 2 次資料庫查詢

```
// Line 70
var totalCount = pharmacies.Count(); // ← 執行一次查詢

// Line 72-84
var details = await pharmacies.Select(...)
    .Skip((page - 1) * pageSize)
    .Take(pageSize)
    .ToListAsync(); // ← 又執行一次查詢
```

可以改成

```
// 先取得資料
var allData = await pharmacies.ToListAsync();

var totalCount = allData.Count;
var details = allData
    .Skip((page - 1) * pageSize)
    .Take(pageSize)
    .ToList();
```

magic code

T4U_Pharmacy_API/T4U_Pharmacy_Service/Service/PurchaseHistoryService.cs

日期的操作有待加強

```
DateTime dtEnd = DateTime.Parse(endDate).AddDays(1);
result.StartDate = dtStart;
result.EndDate = dtEnd.AddDays(-1);
```